





Paths



Unit 2



Lecture 1 - review

- An algorithm – a way to solve a problem or a task
- Should be: Correct, Efficient, Simple
- There may be several algorithms for the same problem.
- There may be several implementations for the same algorithm
- Time and space analysis – using the big-O notation, referring to worst-case complexity.

Lecture 1 - review

GRAPHS:

- $G = (V, E),\ |V| = n, |E| = m.$
- Basic definitions undirected graphs: degree, connected components.
- Basic definitions directed graphs: in-degree, out-degree, strongly connected components.
- Handshaking Theorem (undirected): $\sum_v \deg(v) = 2m$

  Directed graph: $\sum_v indeg(v) = \sum_v outdeg(v) = m.$
- Representations: 1. Adjacency Matrix $O(n^2)$,

  2. Adjacency lists $O(n + m)$.
- A graph is simple if it includes no self-loops or parallel edges.

Lecture 1 - review

Topological sort :

- Input: Directed graph.
- Output: A linear order of the vertices such that if $(u,v) \in E$ then u appears before v.
- TopoSort exists iff the graph is acyclic (DAG)
- Algorithm: In each iteration we add to the output a vertex with in-degree=0, remove it and all edges outgoing from it.
- Time complexity: O(n+m) – if implemented by adj. lists
- Heavily used in the time analysis: In the iteration in which v is processed there are O(out_degree(v)) operations, therefore along the whole execution there are $O(\Sigma_v$ out-deg(v))=m operations.

Path Definition

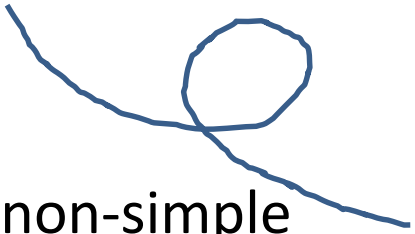
- A path is a sequence of vertices $v_0, v_1, \ldots, v_k$ such that $(vi, v_{i+1}) \in E$ for all $0 \leq i < k$
- v_0 is called the *start vertex* and v_k is called the *end* vertex.
- The *length* of the path is k *(=number of edges).*
- Sometimes, we describe a path as follows:

$$v_0 \xrightarrow{e_1} v_1 \xrightarrow{e_2} \ldots v_{k-1} \xrightarrow{e_k} v_k$$

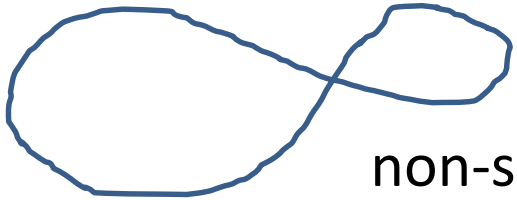
- *It is also possible to list the* edges, that is $e_1, e_2, \ldots, e_k$.

Definitions

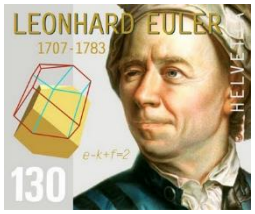
- A *cycle* is a path whose start and end vertices are the same.
- A path is called *simple* if no vertex appears on it more than once.
- A cycle is called *simple* if no vertex, other than the start-end vertex, appears more than once, and the start-end vertex does not appear elsewhere in the cycle.



non-simple path



non-simple cycle



Leonhard Euler

- Euler (1707-1783): a Swiss mathematician . One of the most prolific mathematician of all times (calculus, differential geometry, number theory, etc.)
  - For example, *e*, the natural base for logarithms, is named after him; he introduced the notation f(x), the notation for complex numbers (a *+ i* b) ….
  - Contributions in optics, mechanics, magnetism, electricity
  - "Read Euler, read Euler, he is our master in everything" (Quote from Laplace, a 19th Century French mathematician)

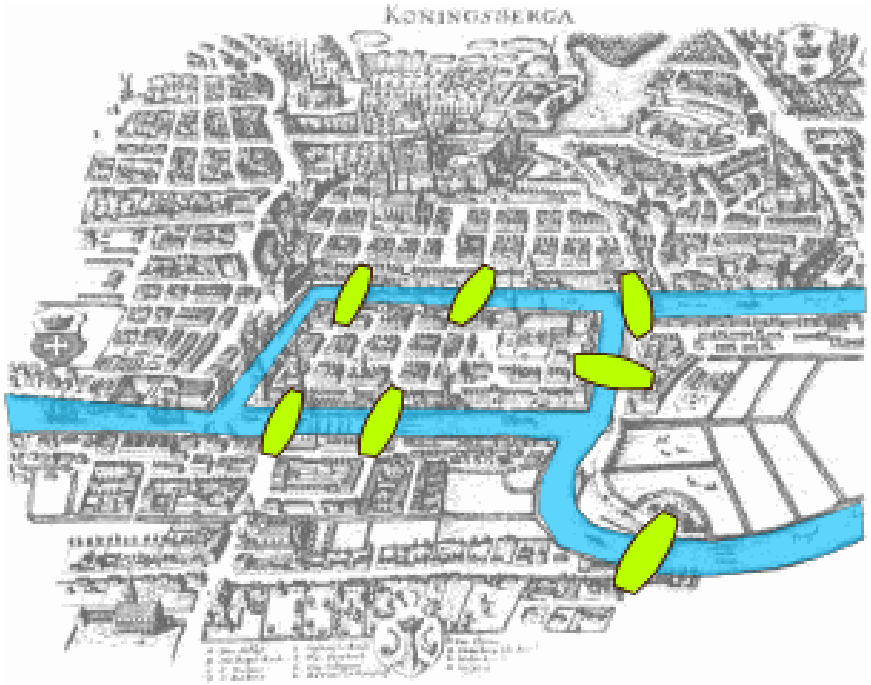




[List of things Named after Euler]

The Seven Bridges Problem

Kaliningrad

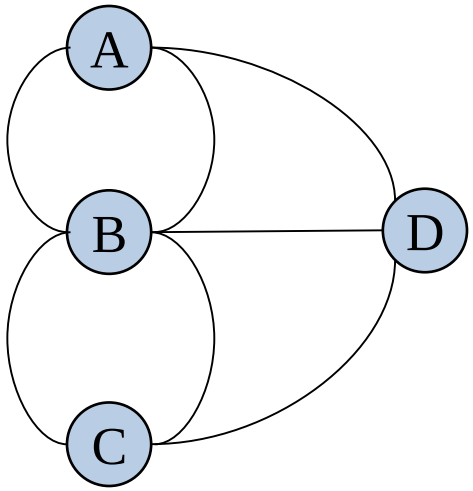


Map of Königsberg in Euler's time showing the actual layout of the seven bridges over river Pregel

Goal: find a way to walk through the city by crossing each bridge exactly once.

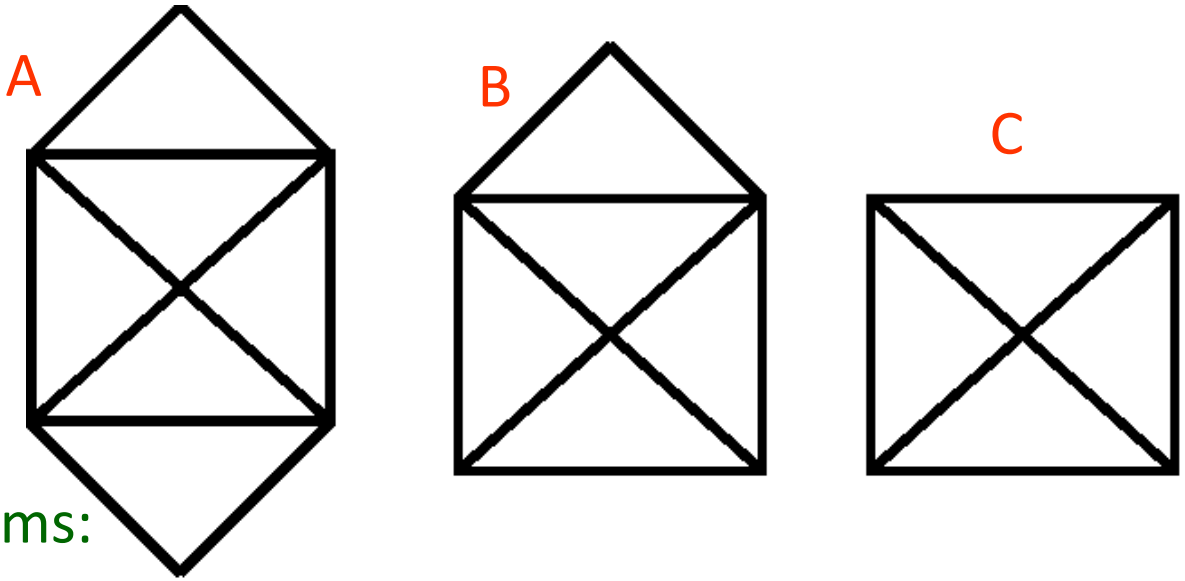
The Seven Bridges Problem

- Each “area” is a vertex
- Each bridge is an edge



Find a path that traverses each edge exactly once

Well-known Puzzles



Two problems:

P1. Can you draw these figures without lifting your pen, drawing each line only once ?

P2. Can you start and end at the same point?

A: P1 - Yes, P2 – Yes.

B: P1 - Yes if you start at lowest right (or left) corner, P2 - No.

C: P1 - No, P2 – No.

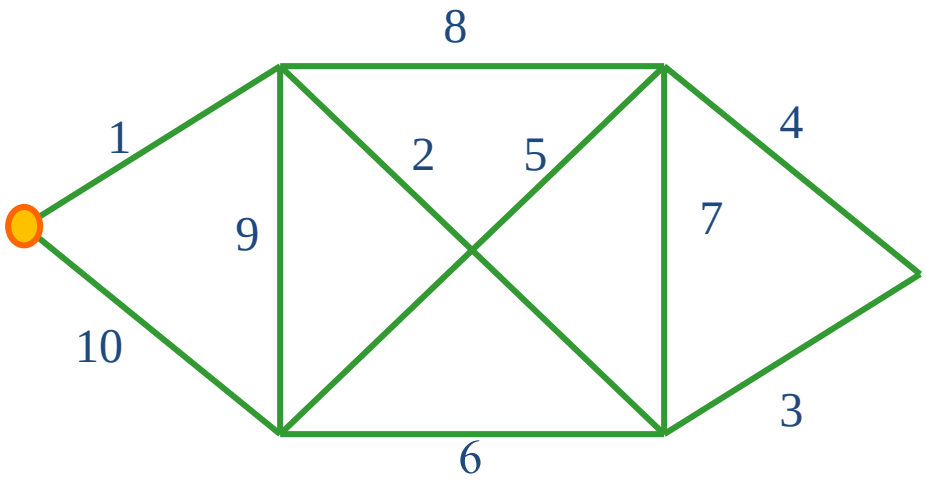
Euler Paths and cycles

- Given G=(V,E), an Euler path is a path that includes each edge exactly once (Problem 1).

- Given G=(V,E) an Euler cycle is an Euler path that starts and ends at the same vertex (Problem 2).

Remark: In this unit, the graphs we consider are not necessarily simple (parallel edges and self-loops allowed)

An Euler cycle for Puzzle A



Each edge is labeled with its index in the path.

Euler cycle Characterization

- A connected graph has an Euler cycle if and only if all its vertices have even degrees
  - Necessary condition (only if): a vertex has to be entered and left whenever it appears in a cycle, hence need an even number of edges at each vertex.
  - Sufficient condition: by construction (linear time algorithm).

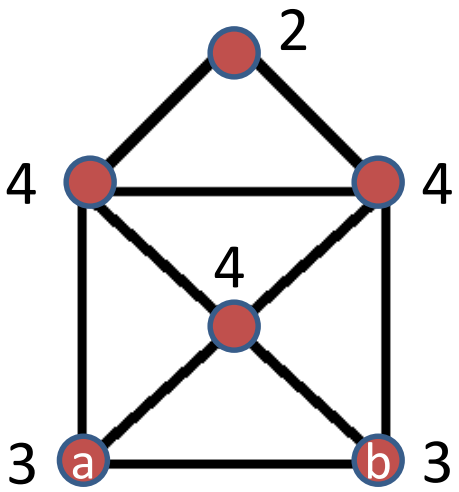
Remark: An unconnected graph has an Euler cycle only if it consists of isolated nodes and one component fulfilling the above conditions.

Euler Path Characterization

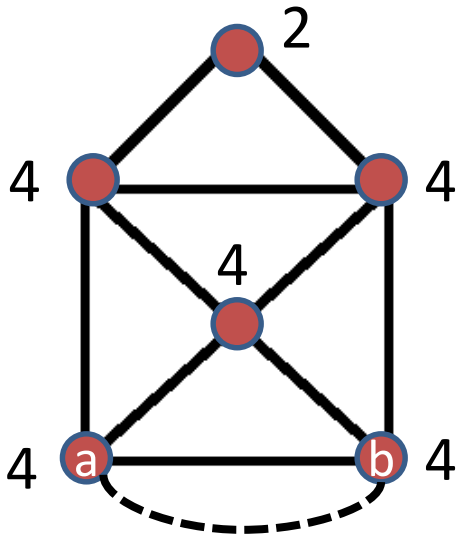
- A connected graph has an Euler path which is not a cycle if and only if exactly two of its vertices have odd degrees.
  - Denote by a and b the odd-degree vertices. One of these vertices will be the start point and the other one will be the end point.
  - To construct the path, add an edge (a,b). Now all vertices have even degrees. Build the Euler cycle and at the end delete the edge (a,b).

Remark: Once again, the graph might be unconnected with one single connected component and other isolated nodes.

Euler Path vs Euler cycle



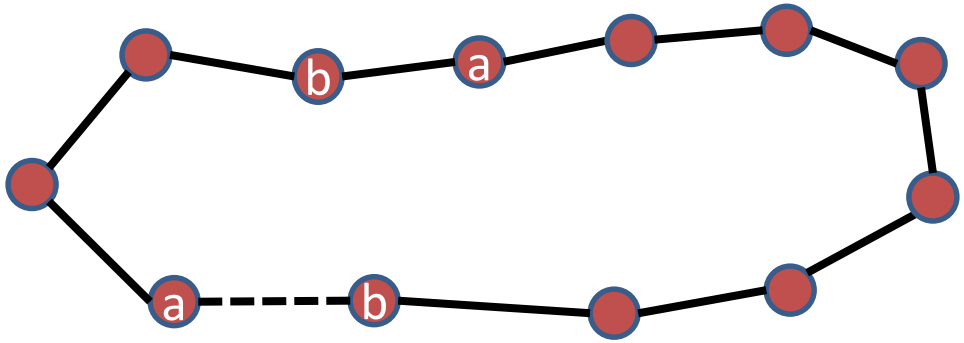
Only a and b have an odd degree



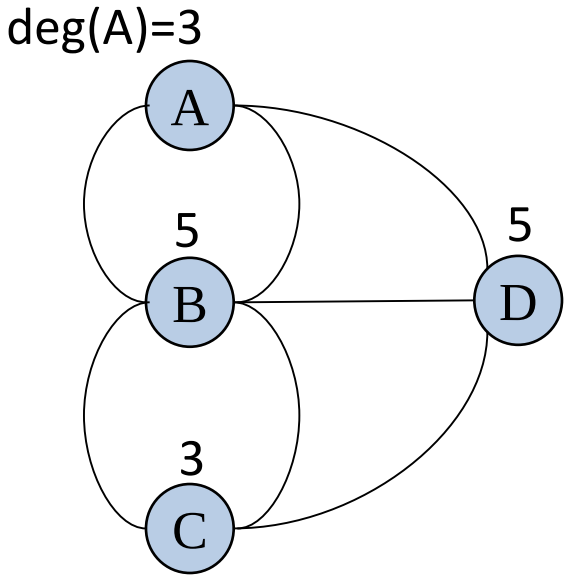
After adding an edge (a,b) all degrees are even. An Euler cycle exists.

Note: the graph may not be simple, it's OK.

Find an Euler cycle. Remove the edge (a,b) to get an Euler path whole endpoints are a and b



Back to Euler Seven Bridges



Sorry, no Euler cycle (not all vertices have even degrees)

Sorry, no Euler path (more than 2 vertices have odd degrees)

Finding an Euler cycle

- Check whether one exists (the graph is connected, and all vertices have even degrees).
- Initialize: all edges are 'untraced' (not included yet in the path)
- Start from an arbitrary vertex A, choose any untraced edge adjacent to it (an edge of which A is an endpoint) and trace it (go to its other endpoint). Upon entering a vertex, search for an untraced incident edge and trace it.
- Repeat as long as possible.

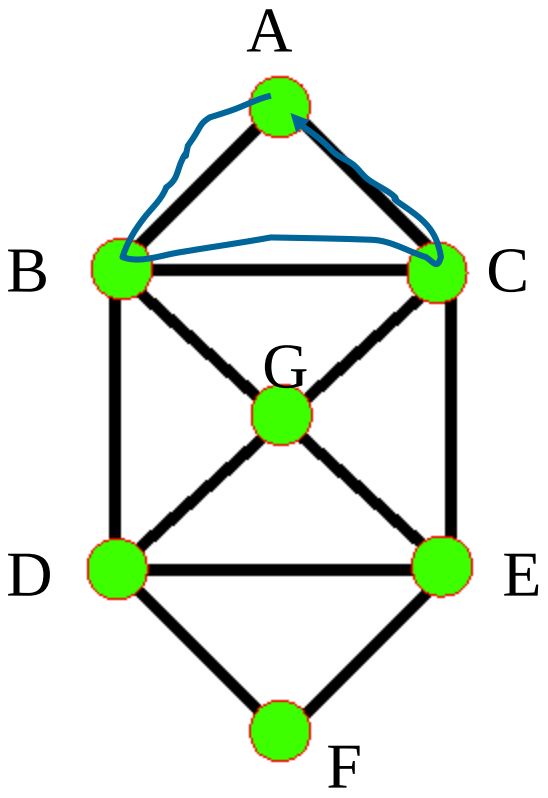
Note: Each time we pass through a vertex we use up two of its incident edges. The degree of the vertex is even. Thus, the number of untraced incident edges after leaving it is even (a self-loop is counted twice).

Finding an Euler cycle (cont')

- The iteration will stop at A (the staring vertex).

  Why?
- So far we have a cycle which starts and ends in A.
- If there are more untraced edges, choose any edge adjacent to the cycle (at least one exists, why?) and build another path of untraced edges. By the same argument, the new path is a cycle, as it must end where it started.
- Merge the cycles to get an extended tour.
- Continue as long as there are untraced edges.

Example

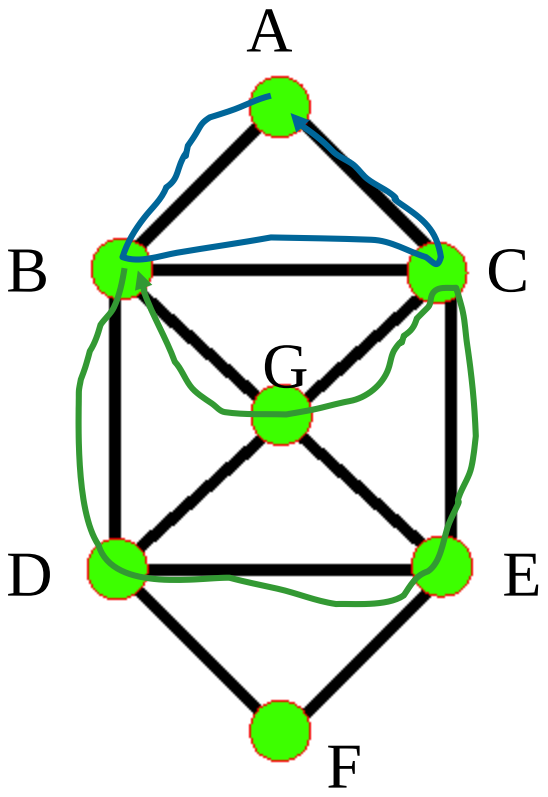


Pick vertex A

Assume the selected path yields the cycle ABCA, that is, edges (A,B),(B,C) and (C,A) are traced,

Pick a vertex in this cycle with an untraced adjacent edge, say B

Example (ct'd)



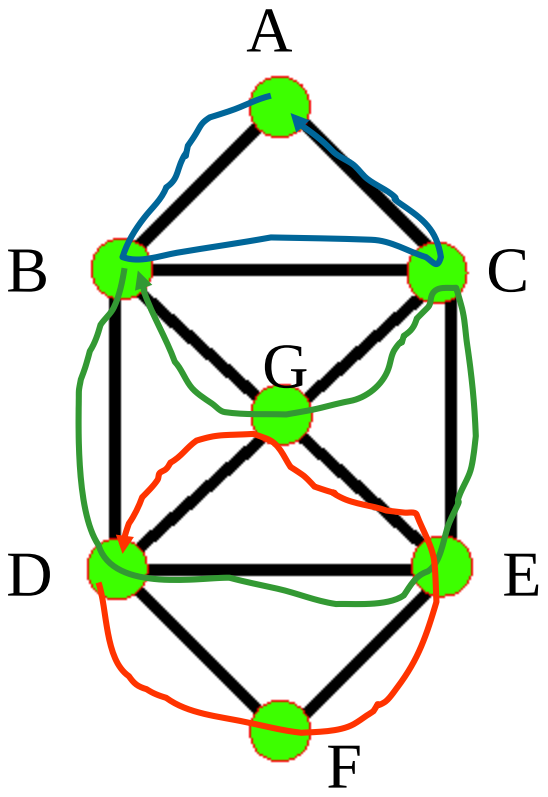
ABCA

Assume picking B yields cycle BDECGB (note that when we reached C, we had to go to G since (C,B), (C,A) and (C,E) are already traced

Merge the green cycle into the blue one

ABDECGBCA

Example (end)



ABDECGBCA

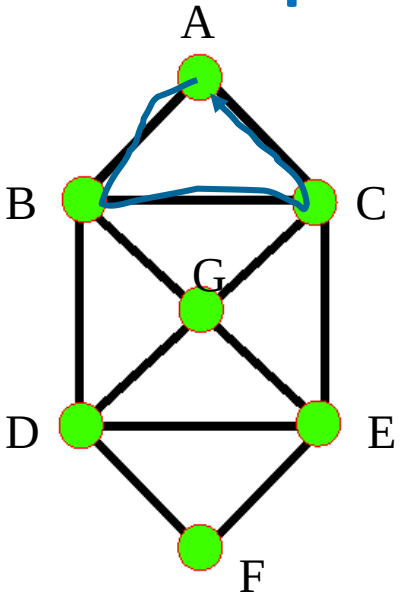
Pick vertex with an adjacent untraced edge- D

Picking D yields DFEGD

Merging yields Euler cycle

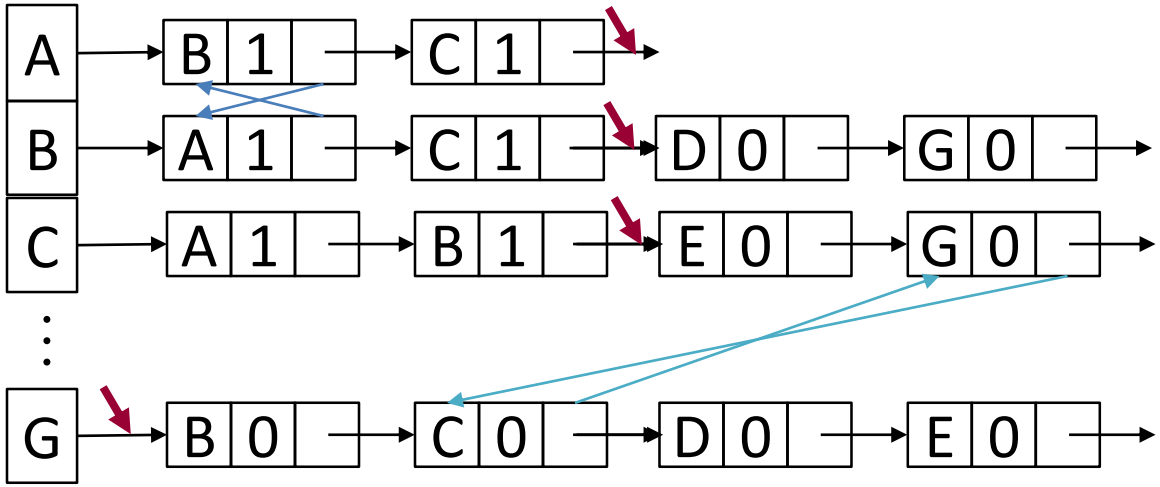
ABDFEGDECGBCA

Implementation in linear time.



Assume the graph is represented by adj. lists.
We should keep track of already-traced edges.
Can be done by adding a 0/1 entry in the corresponding link-list node.
Also add pairs of pointers ⤭ between pairs of 'same edge' entries and a pointer ↘ to next untraced edge in every linked list.

The status after A-B-C-A is traced:



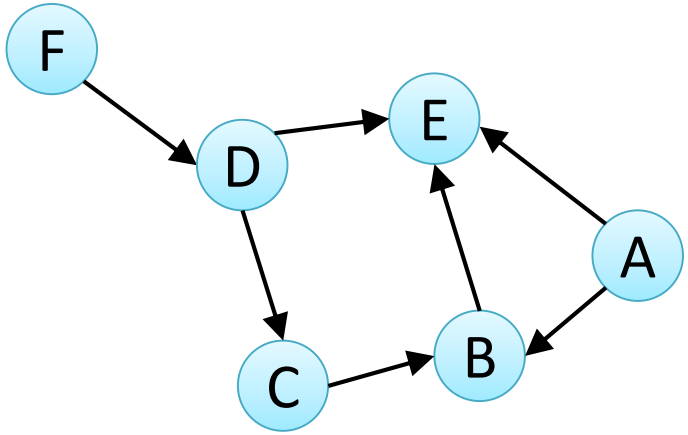
Every list is scanned exactly once along the whole algorithm
⇒ O(n+m)

Euler path in directed graphs

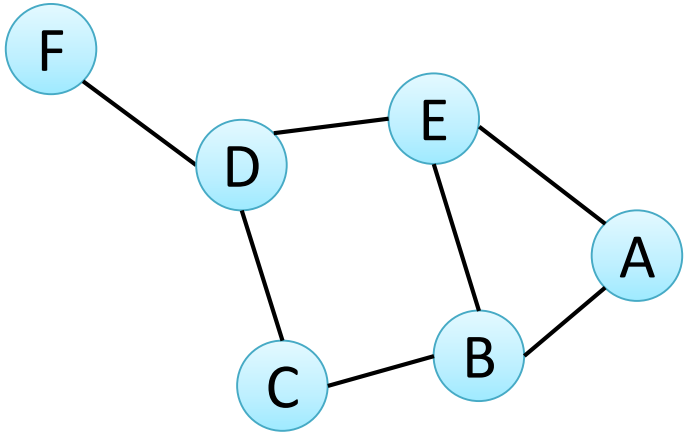
Definition: Let G=(V,E) be a directed graph.

The underlying graph of G is an undirected graph, with the same set of vertices and the same set of edges – without their direction.

Example:



Directed graph G



The underlying graph of G

Euler path in directed graphs

Theorem: A digraph has an Euler path if any only if its underlying graph is connected (except maybe some isolated vertices) and one of the following two conditions holds:

1. For all vertices *v*, $d_{out}(v) = d_{in}(v)$.
2. There is one vertex *a* such that $d_{out}(a) = d_{in}(a) + 1$ and another vertex *b* such that $d_{out}(b) + 1 = d_{in}(b)$, while for all other vertices *v*, $d_{out}(v) = d_{in}(v)$.

If 1 holds then every directed Euler path is a directed Euler cycle.
If 2 holds then every directed Euler path starts in a and ends in *b*.

Algorithm: Identical to undirected case. Edges must be traced according to their direction.

De Bruijn Sequence

A Dutch mathematician 1918-2012

Let $\Sigma = \{0, 1, \ldots, \sigma - 1\}$ be an alphabet of σ letters. There are $L = \sigma^n$ different words of length n over Σ.

A *De Bruijn sequence* is a (circular) sequence $a_0 a_1 \ldots a_{L-1}$ over Σ such that for every word w of length *n* over Σ there exists a unique *i* such that

$$a_i a_{i+1} \ldots a_{i+n-1} = w,$$

where the computation of the indices is modulo L.

- The most important case is of $\sigma = 2$, ($\Sigma = \{0, 1\}$).

Example: A De Bruijn sequence for n=2: 0011

De Bruijn Sequence

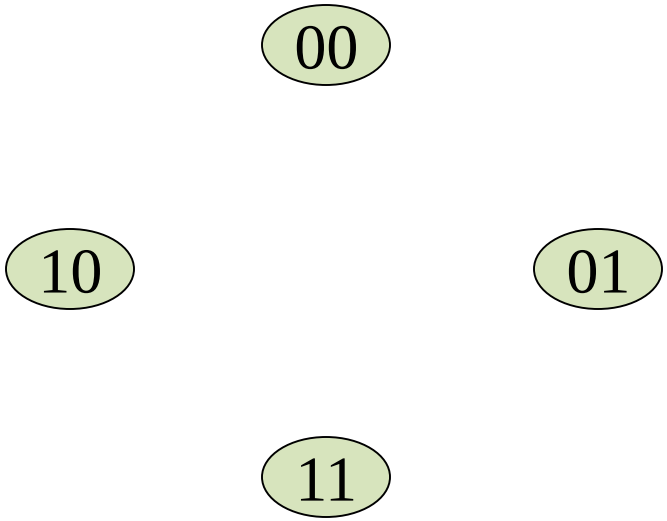
Theorem: For every positive integers σ and n there exists a de Bruijn sequence.

Proof: By construction. In order to find the sequence, build the following digraph $G_{\sigma,n} = (V, E)$:

1. V is the set of all σ^{n-1} words of length $n-1$ over Σ.
2. E is the set of all σ^{n} words of length n over Σ.
3. The edge $b_1 b_2 \ \ldots \ b_n$, starts at vertex $b_1 b_2 \ \ldots b_{n-1}$ and ends at vertex $b_2 b_3 \ \ldots \ b_n$.

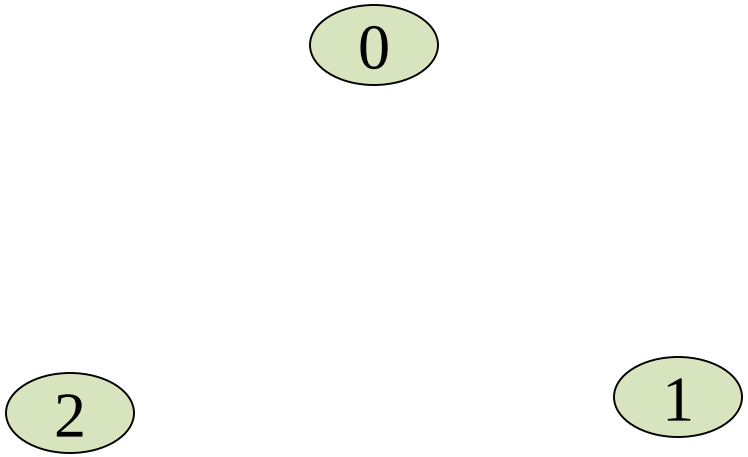
De Bruijn Sequence

Example: G_{23} (complete in class)



De Bruijn Sequence

Example: G_{32} (complete in class)



De Bruijn Sequence

Some properties of these graphs:

1. The word w_2 can follow the word w_1 in a de Bruijn sequence only if the edge w_2 starts at the vertex in which w_1 ends.

2. if the edge w_2 starts at the vertex in which w_1 ends then The word w_2 can follow the word w_1 in a de Bruijn sequence

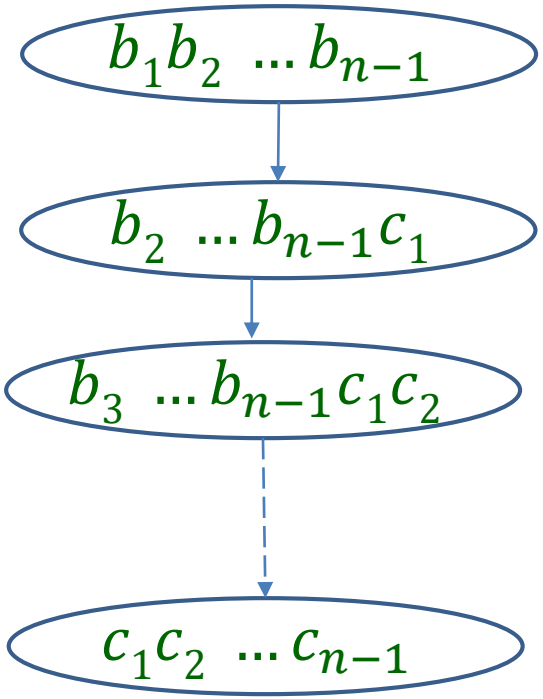
Therefore: A de Bruijn sequences for σ and n exists *if and only if* $G_{\sigma,n}$ has a directed Euler cycle

De Bruijn Sequence

We show that $G_{\sigma,n}$ has an Euler cycle:

1. The underlying undirected graph is connected: Let $b_1 b_2 \ldots b_{n-1}$ and $c_1 c_2 \ldots c_{n-1}$ be any two vertices.

The following directed path leads from the first to the second.



edges

$b_1 b_2 \ldots b_{n-1} c_1$

$b_2 \ldots b_{n-1} c_1 c_2$

$\vdots$

$b_{n-1}\ c_1 c_2 \ldots c_{n-1}$

De Bruijn Sequence

We show that $G_{\sigma,n}$ has an Euler cycle:

2. $d_{out}(v) = d_{in}(v)$ for each vertex *v:* The vertex $b_1 b_2 \ldots b_{n-1}$ is entered by edges $c b_1 b_2 \ldots b_{n-1}$, where c can be chosen in σ ways, and is the start vertex of edges $b_1 b_2 \ldots b_{n-1} c$, where again c can be chosen in σ ways.

⇒ $G_{\sigma,n}$ has an Euler cycle

De Bruijn Sequence

Given the digraph $G_{\sigma,n} = (V, E)$, construct a De-Bruijn sequence in the following way:

1. Find an *Euler Cycle* in $G_{\sigma,n} = (V, E)$.
2. Consider the sequence of edges in the order they appear in the cycle. Take from each edge the 1st letter. The sequence of L letters collected form a De-Bruin sequence.

Example($n = 3, \sigma = 2$):

0̲00, 0̲01, 0̲10, 1̲01, 0̲11, 1̲11, 1̲10, 1̲00

Hamiltonian cycle

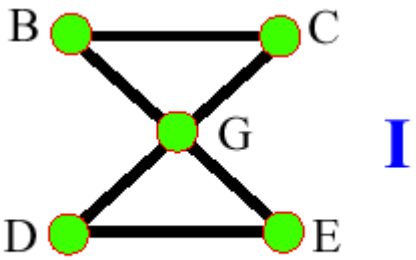
- A Graph G=(V,E) has an Hamiltonian cycle if there exists a cycle which goes through each vertex exactly once.
- Seems closely related to Euler cycle.
- It is NOT!
- Euler cycle can be solved in linear time.
- Hamiltonian cycle requires exhaustive search (exponential time) in the worse case.

Examples

Does Graph I have

an Euler cycle?

a Hamiltonian cycle?

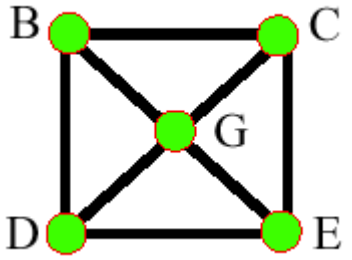


I

Does Graph II have

an Euler cycle?

a Hamiltonian cycle?



II

Finding Hamiltonian cycles

- Apparently an easy “Yes” or “No” question: “Does G contain an Hamiltonian cycle?”
  - No known efficient algorithm, i.e., no algorithm that runs in $O(n^p)$ - polynomial time in the number of vertices.
  - Requires exhaustive search.

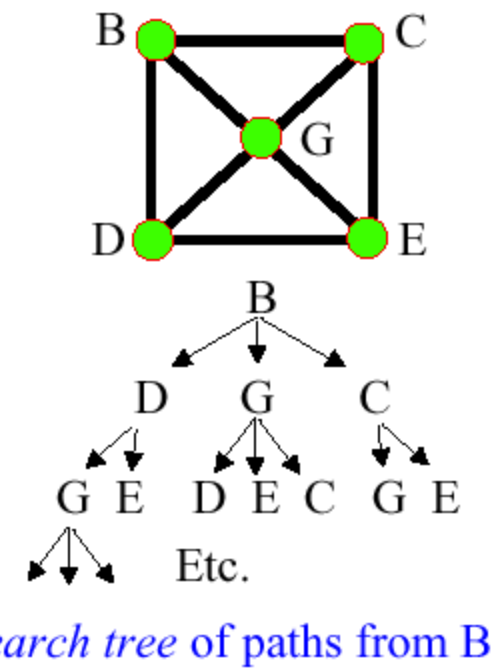
Example of Exhaustive Search

How many candidate paths?

Let r be the average branching factor at each node

Total number of paths to be examined can be estimated by $r \cdot r \cdot r \cdot \ldots \cdot r = O(r^n)$

Exponential time! – not practical.



Search tree of paths from B

Hamiltonian Path and cycle

Sufficient conditions:

undirected

- Dirac Theorem (1952): if $n \geq 3$ and for every vertex v, $d(v) \geq n/2$ then G has a Hamiltonian cycle.
- Ore Theorem (1960): If $n \geq 3$ and for every two vertices v,u in V, $d(u)+d(v) \geq n$ then G has a Hamiltonian cycle.

Note: Dirac's condition is a special case of Ore's condition.

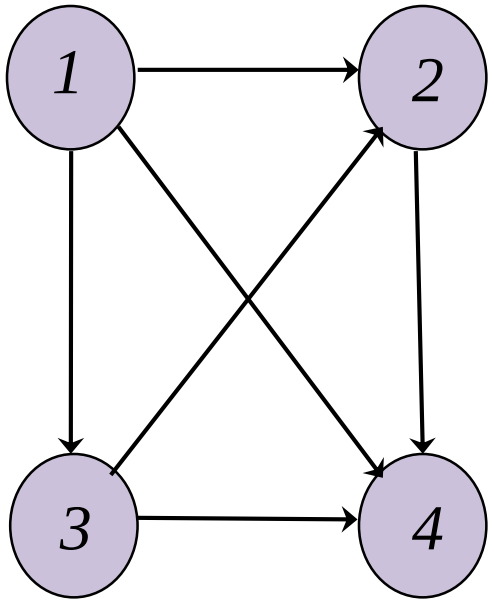
directed

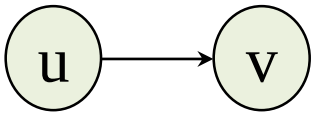
- Every tournament graph has a Hamiltonian path.

A Tournament graph

- N teams
- Input: for each pair of teams, who won the game between them (all pairs of teams competed, no ties)
- Model: a complete graph, each team is represented by a vertex. An edge u→v if u won when playing against v.

Example:



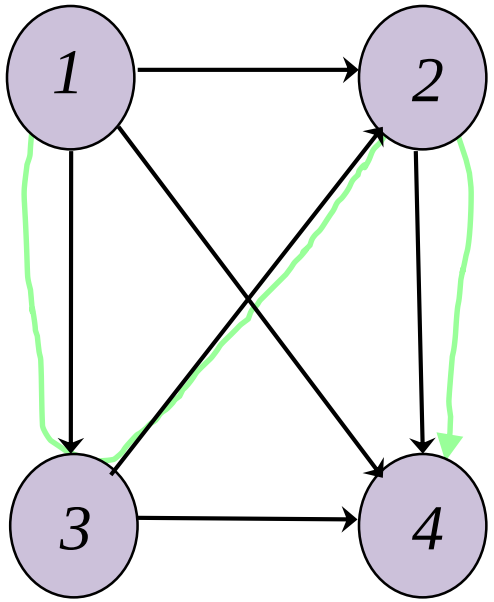


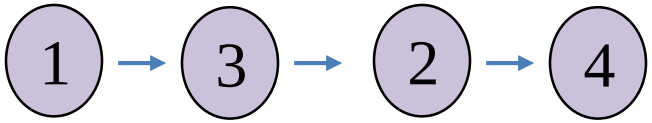
u won against v

A Tournament graph

- A problem: Find an arrangement of the teams on a line, so that every team won in the game against its successor.

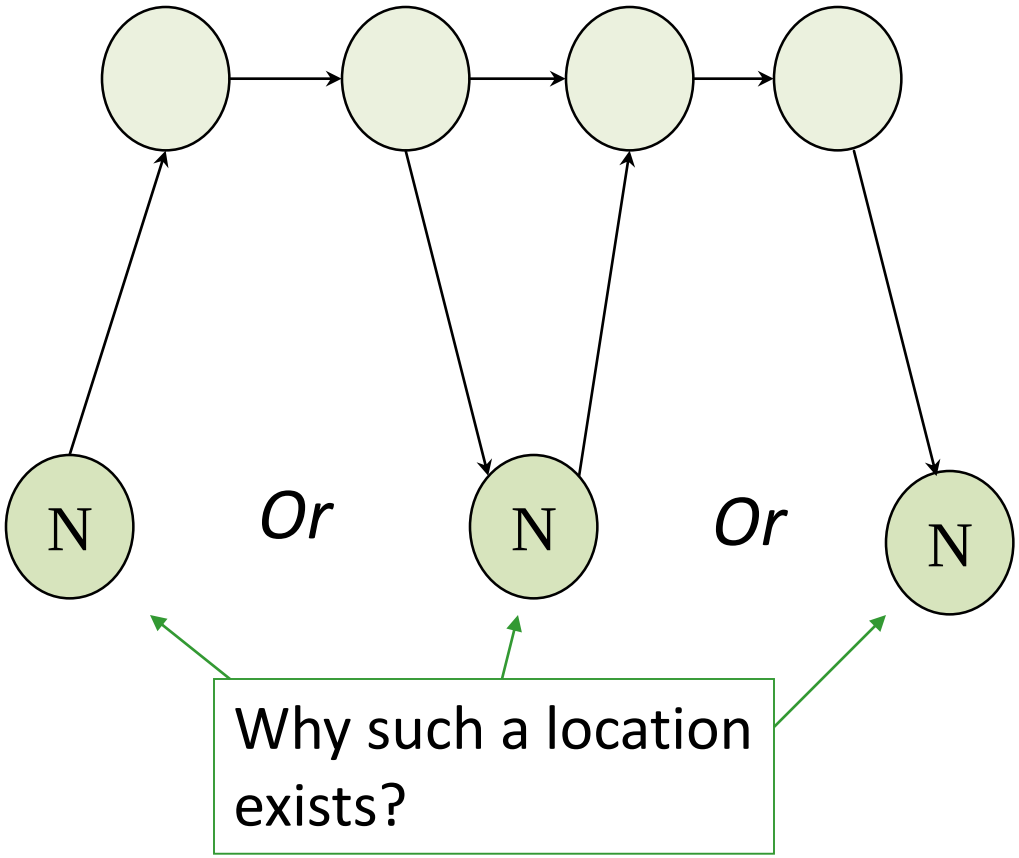
- This is exactly a Hamiltonian path!

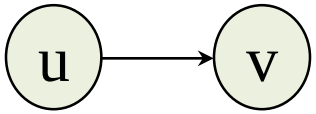




Solution – by induction

- One directed edge define a HamPath.
- Solve the problem for the first N-1 groups
- Find the correct place for the N-th group





u won against v